PERSONAL DESIGN VERIFICATION PROJECT

ASYNCHRONOUS FIFO DESIGN SPECIFICATION

Author: Trần Hồ Khánh Tân
Role: IC Design Student — FPT University

Contents

- 1. Overview
  - 1.1 Introduction
  - 1.2 Applications
  - 1.3 Main Features & Initial Configuration
  - 1.4 Block Diagram & Sub-block Description
  - 1.5 Interface Signals
- 2. Functional Description
  - 2.1 Memory Array (fifomem)
  - 2.2 Write Domain Architecture & wptr Generation
  - 2.3 Read Domain Architecture & rptr Generation
  - 2.4 Clock Domain Crossing (CDC) & 2-Stage Synchronizer
  - 2.5 Gray Code Pointer Arithmetic & CDC Safety
  - 2.6 Flag Generation Logic
  - 2.6.1 Empty Flag Generation (Read Domain)
  - 2.6.2 Full Flag Generation (Write Domain)
  - 2.7 Valid Write and Read Operations
- 3. Verification Methodology
- 4. Corner Cases & CDC Design Rules
- 5. Parameterization Summary
  - 5.1 Core Design Flow
- 6. Document Revision History

List of Tables

- Table 1: Initial configuration parameters
- Table 2: Sub-module partitioning (Cummings CDC architecture)
- Table 3: Interface signals of Asyn_fifo
- Table 4: Binary counter vs. Gray code pointer (FIFO_DEPTH = 8)
- Table 5: Flag generation logic summary
- Table 6: Valid parameter configurations
- Table 7: Document revision history

List of Figures

- Figure 1: Top-level block diagram of Asyn_fifo

1. Overview

1.1 Introduction

- The Asynchronous FIFO (module name: Asyn_fifo) is a parameterizable hardware buffer designed to safely transfer data words between two independent, asynchronous clock domains without data loss, corruption, or metastability issues.

- Unlike a Synchronous FIFO, where both producer and consumer share a single clock and a global element counter, Asyn_fifo operates across two uncoordinated clock domains:

    + Write Domain (wr_clk): use for the write operation .

    + Read Domain (rd_clk): use for the read operation.

- This specification defines the architectural, functional, timing, and verification requirements for the Asyn_fifo design, based on Clifford Cummings' multi-block CDC (Clock Domain Crossing) architecture: Binary Pointer → Gray Pointer → 2-FF Synchronizer → Full/Empty Logic.

1.2 Applications

- Clock Domain Crossing (CDC): bridging high-speed and low-speed processing islands.

- Elastic buffering: absorbing burst transactions between interfaces with varying or drifting frequency/phase relationships.

- Rate matching: decoupling processor pipelines from external peripherals.

1.3 Main Features & Initial Configuration

- Independent dual clocks: separate wr_clk and rd_clk with arbitrary frequency and phase relationships.

- Parameterizable data width (DATA_WIDTH) and depth (FIFO_DEPTH = 2^ADDR_WIDTH): power-of-two depth for glitch-free Gray-code roll-over.

- Independent asynchronous resets: dedicated active-low reset for each domain (wr_rst_n, rd_rst_n).

- Glitch-free pointer synchronization: pointers converted to Gray code before 2-stage D-flip-flop synchronizers.

- Pessimistic, safe flag assertion: full (write domain) and empty (read domain) are conservative but never unsafe .

- Dual-port RAM core: synchronous write, registered read.

Initial Parameter Configuration

The following configuration is chosen as the initial implementation:

```
DATA_WIDTH = 8
FIFO_DEPTH = 8
ADDR_WIDTH = 3
PTR_WIDTH  = 4
```

Parameter relationship rules:

```
FIFO_DEPTH = 2^ADDR_WIDTH
PTR_WIDTH  = ADDR_WIDTH + 1
```

Parameter	Initial Value	Description
DATA_WIDTH	8	FIFO data width
ADDR_WIDTH	3	Address width
FIFO_DEPTH	8	Number of FIFO entries
PTR_WIDTH	4	Pointer width (ADDR_WIDTH + 1)

Table 1: Initial configuration parameters

- This configuration is used as the initial implementation and verification configuration. The RTL is parameterized to support other power-of-two FIFO depths and configurable data widths.

Note: 8-bit × 8-depth is only the initial configuration used for bring-up and verification — it is not a limitation of the design.

1.4 Block Diagram & Sub-block Description

- To keep the block diagram consistent with the RTL and with the rest of this document, the diagram must follow these naming conventions:

  + FIFO memory block labeled with the actual parameters, e.g. "DATA_WIDTH = 8, FIFO_DEPTH = 8" (or "8 bits × 8 words") — not generic "W" / "L".

  + Write Pointer block labeled with both representations: Binary [3:0] and Gray [3:0].

  + Read Pointer block labeled with both representations: Binary [3:0] and Gray [3:0].

  + Address signals labeled wr_addr[2:0] and rd_addr[2:0].

  + Clock/reset signals labeled only as wr_clk, rd_clk, wr_rst_n, rd_rst_n.

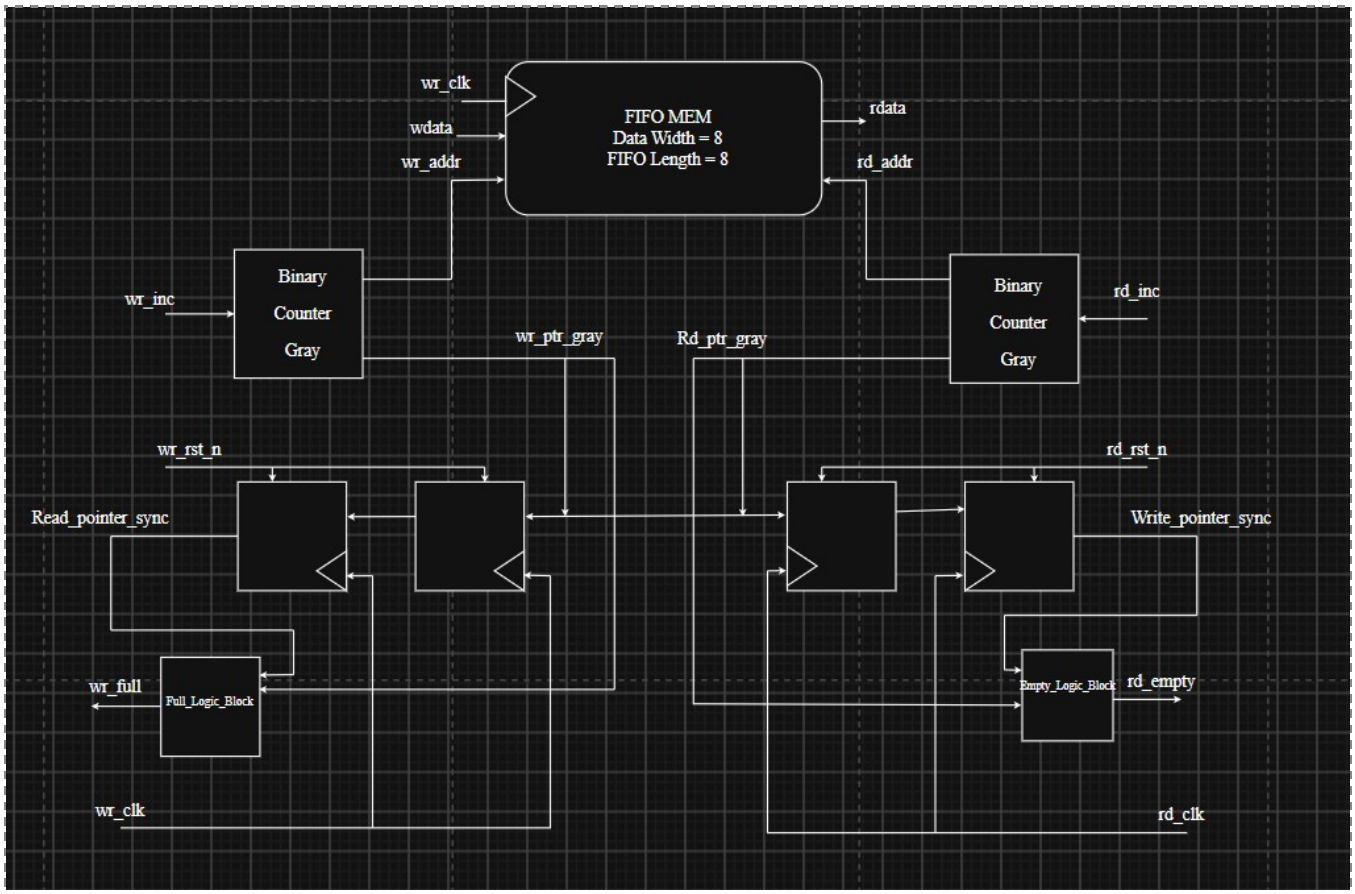


Figure 1 — Top-level block diagram of Asyn_fifo]

Internal Architecture Partitioning (Cummings CDC Paradigm)

To follow standard ASIC/FPGA synthesis and CDC static-verification practices, Asyn_fifo is hierarchically partitioned into the following sub-modules:

Sub-module	Function	Clock Domain
fifomem	Dual-port memory array of depth FIFO_DEPTH and	wr_clk (write side)

Sub-module	Function	Clock Domain
	width DATA_WIDTH. Writes synchronous to wr_clk; reads indexed by read address.	
sync_r2w	2-stage flip-flop synchronizer passing the read Gray pointer (rd_ptr) into wr_clk domain, producing wq2_rdptr.	wr_clk
sync_w2r	2-stage flip-flop synchronizer passing the write Gray pointer (wrptr) into rd_clk domain, producing rq2_wrptr.	rd_clk
wr_ptr_full	Binary write counter, binary-to-Gray converter, and full-condition comparator.	wr_clk
rd_ptr_empty	Binary read counter, binary-to-Gray converter, and empty-condition comparator.	rd_clk

Table 2: Sub-module partitioning (Cummings CDC architecture)

1.5 Interface Signals

Signal	Width	Direction	Domain	Description
wr_clk	1	Input	Write	Write clock
wr_rst_n	1	Input	Write	Active-low write reset
wr_inc	1	Input	Write	Write request
wdata	DATA_WIDTH	Input	Write	Write data
wr_full	1	Output	Write	FIFO full flag
rd_clk	1	Input	Read	Read clock
rd_rst_n	1	Input	Read	Active-low read reset
rd_inc	1	Input	Read	Read request
rdata	DATA_WIDTH	Output	Read	Read data
rd_empty	1	Output	Read	FIFO empty flag

Table 3: Interface signals of Asyn_fifo

2. Functional Description

2.1 Memory Array (fifomem)

- Organized as FIFO_DEPTH = 2^ADDR_WIDTH words of DATA_WIDTH bits each.
- Write address: wr_addr = wr_bin[ADDR_WIDTH-1:0].
- Read address: rd_addr = rd_bin[ADDR_WIDTH-1:0].
- Write: memory write occurs on the rising edge of wr_clk when write_accept = wr_inc && !wr_full.
- Read: rdata is registered on the rising edge of rd_clk when a valid read operation is accepted (read_accept = rd_inc && !rd_empty).

2.2 Write Domain Architecture & wr_ptr Generation

- (ADDR_WIDTH+1)-bit binary counter (wr_bin) tracks writes; the extra MSB tracks wrap-around cycles to distinguish Full from Empty.

- On each wr_clk edge: if wr_inc == 1 and wr_full == 0, wr_bin <= wr_bin + 1.

- wr_binnext = wr_bin + (wr_inc && !wr_full).

- Gray transform: wr_gnext = (wr_binnext >> 1) ^ wr_binnext.

- wr_ptr register stores wr_gnext on each wr_clk edge.

- Signal widths (for FIFO_DEPTH = 2^ADDR_WIDTH):

    + wr_bin, wr_ptr: ADDR_WIDTH + 1 bits.

    + Write address: wr_bin[ADDR_WIDTH-1:0].

-Initial configuration (ADDR_WIDTH = 3):

```
wr_bin[3:0]
wptr[3:0]
write_addr[2:0]
```

2.3 Read Domain Architecture & rptr Generation

- (ADDR_WIDTH+1)-bit binary counter (rd_bin) tracks reads.

- On each rclk edge: if rd_inc == 1 and rd_empty == 0, rd_bin <= rd_bin + 1.

- rd_binnext = rd_bin + (rd_inc && !rd_empty).

- Gray transform: rd_gnext = (rd_binnext >> 1) ^ rd_binnext.

- rd_ptr register stores rd_gnext on each rd_clk edge.

- Signal widths (for FIFO_DEPTH = 2^ADDR_WIDTH):

    + rd_bin, rd_ptr: ADDR_WIDTH + 1 bits.

    + read address: rd_bin[ADDR_WIDTH-1:0].

Initial configuration (ADDR_WIDTH = 3):

```
rd_bin[3:0]
rptr[3:0]
read_addr[2:0]
```

2.4 Clock Domain Crossing (CDC) & 2-Stage Synchronizer

- Transferring multi-bit binary values directly across asynchronous domains is prone to multi-bit reconvergence faults, since interconnect delays cause bits to arrive at different clock edges.

- To guarantee robust operation:

    + Gray encoding — pointers change by exactly 1 bit per increment.

    + 2-stage flip-flop synchronizer — removes metastability by allowing a full clock period for a metastable first stage output to settle before being sampled by the second stage.

- Only the Gray-coded pointer crosses the clock-domain boundary. The binary pointer remains local to its own clock domain and is never synchronized directly:

```
wr_bin (binary)
  └── local to write domain only

wr_ptr (Gray-coded write pointer)
  └── 2-FF synchronizer ──> read domain, producing rq2_wrptr

rd_bin (binary)
  └── local to read domain only
```

```
rd_ptr (Gray-coded read pointer)
  └── 2-FF synchronizer ──> write domain, producing wq2_rdptr
```

2.5 Gray Code Pointer Arithmetic & CDC Safety

For the initial configuration (FIFO_DEPTH = 8, ADDR_WIDTH = 3, PTR_WIDTH = 4):

Decimal	Binary	Gray	Bit Changes vs Previous
0	0000	0000	—
1	0001	0001	1 bit (bit 0)
2	0010	0011	1 bit (bit 1)
3	0011	0010	1 bit (bit 0)
4	0100	0110	1 bit (bit 2)
5	0101	0111	1 bit (bit 0)
6	0110	0101	1 bit (bit 1)
7	0111	0100	1 bit (bit 0)
8 (wrap boundary)	1000	1100	1 bit (bit 3)

Table 4: Binary counter vs. Gray code pointer (FIFO_DEPTH = 8)

- The Gray pointer changes by only one bit between consecutive pointer values, including the transition across the FIFO wrap point (decimal 7 → 8).

2.6 Flag Generation Logic

Feature	Asyn_fifo (This Design)
Pointer tracking	Gray code pointers + binary
Shared counter	No shared counter
Empty detection	rd_ptr == rq2_wptr
Full detection	wr_ptr == {~wq2_rdptr[N:N-1], wq2_rdptr[N-2:0]}

2.6.1 Empty Flag Generation (Read Domain)

- The FIFO is empty when the read pointer catches up to the (synchronized) write pointer in the read domain. rd_empty is registered synchronously to rd_clk.

- The read domain can determine that the FIFO is empty only after the write pointer has been synchronized into the read domain through the 2-FF synchronizer.

2.6.2 Full Flag Generation (Write Domain)

- The FIFO is full when the write pointer has wrapped exactly one more cycle than the synchronized read pointer. In (N+1)-bit Gray code, the full condition is:

    + MSB of wr_ptr and wq2_rdptr differ (write has wrapped one more cycle).
    + Second MSB (bit N-1) of wr_ptr and wq2_rdptr differ (Gray-code mirror symmetry).
    + All remaining lower address bits (N-2 down to 0) are identical.
    + wr_full is registered synchronously to wr_clk.

```
wr_ptr
  |
  | compare with
  v
modified synchronized rd_ptr (wq2_rdptr with MSBs inverted)
  |
  v
wr_full
```

Flag	Clock Domain	Condition / Expression	Type
rd_empty	rd_clk	rd_gnext == rq2_wrptr	Registered
wr_full	wr_clk	wr_gnext == {~wq2_rdptr[N:N-1], wq2_rdptr[N-2:0]}	Registered

Table 5: Flag generation logic summary

2.7 Valid Write and Read Operations

- A write operation is accepted only when:

```
write_accept = wr_inc && !wr_full
```

- A read operation is accepted only when:

```
read_accept = rd_inc && !rd_empty
```

- When write_accept is asserted:

  + The memory write is performed .
  + The write pointer (wr_bin / wr_ptr) advances .

- When read_accept is asserted:

  + The read operation is performed and rdata is updated.
  + The read pointer (rd_bin / rd_ptr) advances .

- Requests asserted while the corresponding FIFO boundary flag is active (wr_inc while wr_full, or rd_inc while rd_empty) are not accepted: no memory update, no pointer update. write_accept and read_accept are the reference signals used throughout verification and should be tracked directly by the scoreboard.

3. Verification Methodology

- This section provides a high-level overview of the verification scope for the Asynchronous FIFO design.

- The detailed verification strategy, test cases, stimulus, expected behavior, pass/fail criteria, waveform analysis, scoreboard checking, assertions, and coverage requirements are defined in the Verification Plan.

- The main verification objectives are:

  + Verify correct FIFO write and read functionality.

  + Verify correct Full and Empty flag behavior.

  + Verify data integrity and FIFO ordering.

+ Verify protection against overflow and underflow.

+ Verify correct operation between asynchronous clock domains.

+ Verify reset behavior and pointer initialization.

+ Verify FIFO behavior under different clock frequency and phase relationships.

+ Verify boundary transitions between Full, Normal, and Empty states.

4. Corner Cases & CDC Design Rules

- Write overflow prevention: if wr_inc = 1 while wr_full = 1, the write request is discarded; neither fifomem nor wr_bin/wr_ptr update.
- Read underflow prevention: if rd_inc = 1 while rd_empty = 1, the read request is discarded; rd_bin/rd_ptr do not increment and rdata retains its prior value.
- Power-of-two depth constraint (FIFO_DEPTH = 2^ADDR_WIDTH): depth must strictly be a power of 2 (4, 8, 16, 32, ...) so gray code rolls over from 2^(N+1)-1 to 0 with only a 1-bit transition. Non-power-of-2 depths require specialized truncated Gray counters and are out of scope for Version 1.
- Synchronizer placement & boundary constraint: no combinational logic is permitted between the Gray-counter output registers and the 2-FF synchronizer inputs, to avoid propagating combinational glitches across domains.
- Reset synchronization: wr_rst_n and rd_rst_n must be asynchronously asserted and synchronously deasserted within their respective clock domains, to eliminate reset removal/recovery timing violations.
- wr_clk and rd_clk may have arbitrary frequency and phase relationships; the design must not assume any fixed ratio between them.
- Write and read requests may occur simultaneously; neither domain's logic may assume the other is idle.
- FIFO ordering must be preserved: data is read in the same order in which valid writes were accepted (write_accept order).
- Invalid write requests while wr_full = 1 must not modify memory or the write pointer.
- Invalid read requests while rd_empty = 1 must not advance the read pointer or perform a valid read.
- A reset event invalidates the current FIFO contents unless an explicit system-level recovery mechanism preserves them .

5. Parameterization Summary

- Initial configuration:

```
DATA_WIDTH = 8
FIFO_DEPTH = 8
ADDR_WIDTH = 3
PTR_WIDTH  = 4
```

- The RTL must generalize to any configuration satisfying:

```
FIFO_DEPTH = 2^ADDR_WIDTH
PTR_WIDTH  = ADDR_WIDTH + 1
```

DATA_WIDTH	FIFO_DEPTH	ADDR_WIDTH	PTR_WIDTH
8	8	3	4
8	16	4	5

DATA_WIDTH	FIFO_DEPTH	ADDR_WIDTH	PTR_WIDTH
16	16	4	5
32	32	5	6
32	64	6	7

Table 6: Valid parameter configurations

5.1 Core Design Flow

```
Parameters
    |
FIFO Memory
    |
Binary Write/Read Pointer
    |
Gray Conversion
    |
2-FF Synchronizer
    |
Cross-domain Pointer
    |
Full / Empty Logic
    |
Valid Write / Read
    |
Verification
    |
Scoreboard
    |
Assertions
    |
Coverage
```

6. Document Revision History

Date	Version	Description	Author
[19/09/2026]	0.1	Initial draft: architecture, multi-clock domain analysis, block diagram, interface signals.	Trần Hồ Khánh Tân
[20/09/2026/2026]	0.2	Added Gray code arithmetic, 2-stage synchronizer details, pessimistic full/empty equations.	Trần Hồ Khánh Tân
[20/09/2026]	0.3	Finalized CDC corner cases, design rules, and modular sub-block specification and the rest of the design specification	Trần Hồ Khánh Tân

Table 7: Document revision history